🧪 Lab3 — MIME File Transfer over Router-on-a-Stick with Wireshark Analysis

A Cisco Networking Academy–style lab that blends classical networking with application-layer engineering. You will configure inter-VLAN routing, deploy a MIME-aware socket server, capture packets, and analyze how meaning flows through the stack.

1. นายกรกฏ พรมทอง 673380025-8 Sec2

2.นายสรวิชญ์ ทะมานันท์ 673380295-9 Sec2

3.นายวัชรวิศว์ น้อยเมล์ 673380059-1 Sec2

📘 Lab Overview

Scenario

A client in one VLAN transfers files to a server in another VLAN using a custom MIME-based protocol (JSON header + binary payload). You will verify connectivity, observe encapsulation, and analyze packets using Wireshark.

This lab reinforces how Layer 2 segmentation, Layer 3 routing, TCP reliability, and application metadata interact.

🎯 Objectives

After completing this lab, you will be able to:

- Configure Router-on-a-Stick inter-VLAN routing
- Configure VLANs and trunking on a switch
- Deploy and run a MIME socket server and client
- Capture traffic using Wireshark
- Identify ARP, VLAN tags, TCP handshake, and application data
- Decode JSON headers in packet payload
- Troubleshoot network and application failures
- Document protocol behavior across OSI layers

🧰 Required Resources

- 1 Router (e.g., Cisco Systems ISR/virtual router)
- 1 Switch (Layer 2)
- 2 PCs or VMs
- Ethernet cables or virtual links
- Python 3 installed
- MIME file transfer project
- Wireshark installed (from Wireshark Foundation)

🗺️ Topology

```
None
PC1 (VLAN 10) ----\
                    \ access
                     \
                     [S1]---- trunk ----[R1]
                     /
                    / access
PC2 Server (VLAN 20)
```

📋 Addressing Scheme

Device	Interface	IP	Gateway
R1	G0/0.10	192.168.10.1/24	—
R1	G0/0.20	192.168.20.1/24	—
PC1-Client	NIC	192.168.10.10	192.168.10.1
PC2-Server	NIC	192.168.20.20	192.168.20.1

🧠 Background Theory

Router-on-a-Stick

A single physical interface carries multiple VLANs via 802.1Q tagging. Subinterfaces perform routing between broadcast domains.

MIME Metadata

MIME labels content type (e.g., image/png). Your protocol wraps files with JSON describing:

- MIME type
- Size

Why This Matters

Real systems — APIs, storage services, message brokers — depend on this exact pattern: structured metadata + binary payload over reliable transport.

🔧 Part 1 — Configure Network Devices

Step 1 — Router Configuration

```
None
enable
configure terminal
hostname R1

interface g0/0/0
no shutdown

interface g0/0/0.10
encapsulation dot1Q 10
ip address 192.168.10.1 255.255.255.0

interface g0/0/0.20
encapsulation dot1Q 20
ip address 192.168.20.1 255.255.255.0

end
write memory
```

```
Router>en
Router#config t
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#hostname R1
R1(config)#interface g0/0/0
R1(config-if)#no shutdown

R1(config-if)#
%LINK-5-CHANGED: Interface GigabitEthernet0/0/0, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0, changed state to up

R1(config-if)#interface g0/0/0.10
R1(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0/0.10, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0.10, changed state to up

R1(config-subif)#encapsulation dot1Q 10
R1(config-subif)#ip address 192.168.10.1 255.255.255.0
R1(config-subif)#interface g0/0/0.20
R1(config-subif)#
%LINK-3-UPDOWN: Interface GigabitEthernet0/0/0.20, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/0/0.20, changed state to up

R1(config-subif)#encapsulation dot1Q 20
R1(config-subif)#ip address 192.168.20.1 255.255.255.0
R1(config-subif)#end
R1#
%SYS-5-CONFIG_I: Configured from console by console

R1#write memory
Building configuration...
[OK]
R1#
```

Step 2 — Switch Configuration

```
None
enable
configure terminal
hostname S1

vlan 10
name CLIENTS

vlan 20
name SERVERS

interface g0/1
switchport mode trunk

interface fa0/1
switchport mode access
switchport access vlan 10

interface fa0/2
switchport mode access
switchport access vlan 20

end
```

```
write memory
```

```
Switch>en
Switch#config t
Enter configuration commands, one per line.  End with CNTL/Z.
Switch(config)#hostname S1
S1(config)#vlan 10
S1(config-vlan)#name CLIENTS
S1(config-vlan)#vlan 20
S1(config-vlan)#name SERVERS
S1(config-vlan)#
```

```
S1(config-vlan)#
S1(config-vlan)#interface g0/1
S1(config-if)#switchport mode trunk

S1(config-if)#
%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to down

%LINEPROTO-5-UPDOWN: Line protocol on Interface GigabitEthernet0/1, changed state to up

S1(config-if)#interface fa0/1
S1(config-if)#switch mode access
S1(config-if)#switchport access vlan 10
S1(config-if)#interface fa0/2
S1(config-if)#switchport mode access
S1(config-if)#switctport access vlan 20
                ^
% Invalid input detected at '^' marker.

S1(config-if)#end
S1#
%SYS-5-CONFIG_I: Configured from console by console

S1#
```

```
S1#write memory
Building configuration...
[OK]
S1#
```

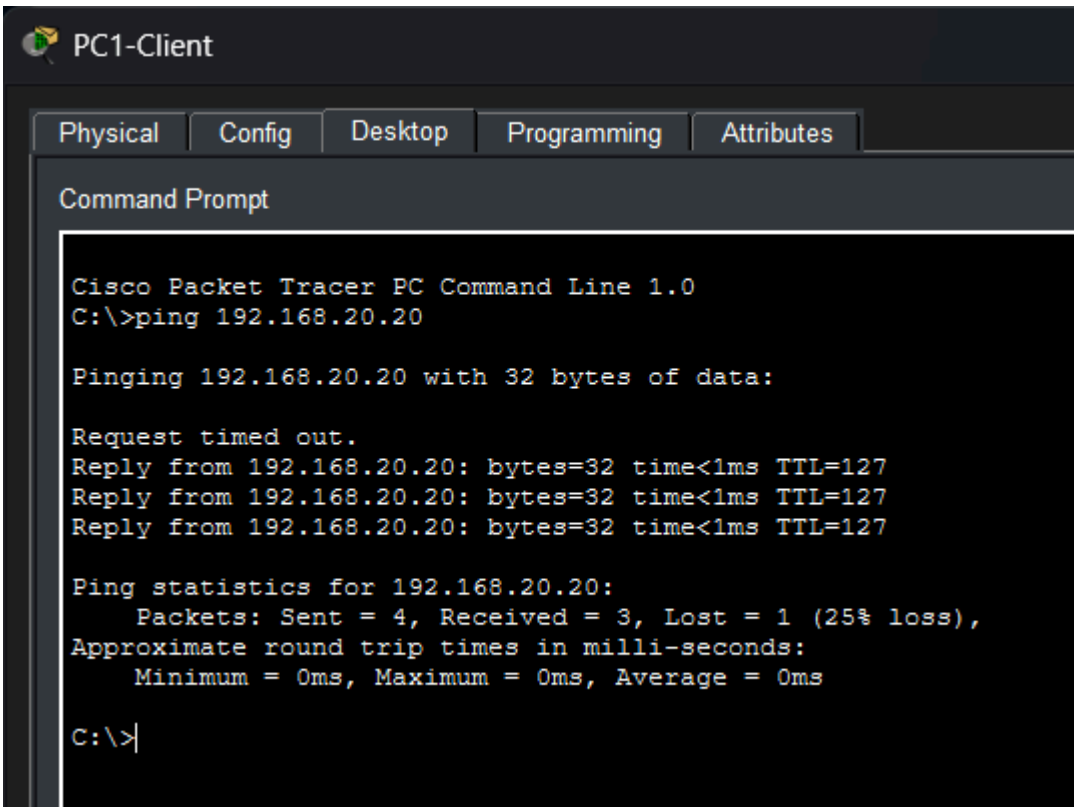
Step 3 — Configure Hosts

Set static IP addresses per table.

🔍 Part 2 — Verify Connectivity

From PC1:

```
None
ping 192.168.20.20
```



Verify:

- Successful replies
- ARP entries

Router checks:

```
None
show ip route
show ip interface brief
```

```
R1>show ip route
Codes: L - local, C - connected, S - static, R - RIP, M - mobile, B - BGP
       D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area
       N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2
       E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP
       i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area
       * - candidate default, U - per-user static route, o - ODR
       P - periodic downloaded static route

Gateway of last resort is not set

     192.168.10.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.10.0/24 is directly connected, GigabitEthernet0/0/0.10
L       192.168.10.1/32 is directly connected, GigabitEthernet0/0/0.10
     192.168.20.0/24 is variably subnetted, 2 subnets, 2 masks
C       192.168.20.0/24 is directly connected, GigabitEthernet0/0/0.20
L       192.168.20.1/32 is directly connected, GigabitEthernet0/0/0.20

R1>show ip interface brief
Interface              IP-Address      OK? Method Status                Protocol
GigabitEthernet0/0/0   unassigned      YES unset  up                    up
GigabitEthernet0/0/0.10192.168.10.1    YES manual up                    up
GigabitEthernet0/0/0.20192.168.20.1    YES manual up                    up
GigabitEthernet0/0/1   unassigned      YES unset  down                  down
Vlan1                  unassigned      YES unset  up                    down
R1>
```

Switch checks:

```
None
show interfaces trunk
show mac address-table
```

```
S1>show interfaces trunk
Port        Mode         Encapsulation  Status        Native vlan
Gig0/1      on           802.1q         trunking      1

Port        Vlans allowed on trunk
Gig0/1      1-1005

Port        Vlans allowed and active in management domain
Gig0/1      1,10,20

Port        Vlans in spanning tree forwarding state and not pruned
Gig0/1      1,10,20

S1>show mac address-table
          Mac Address Table
-------------------------------------------

Vlan    Mac Address       Type        Ports
----    -----------       --------    -----

   1    0009.7c55.7a01    DYNAMIC     Gig0/1
  10    0009.7c55.7a01    DYNAMIC     Gig0/1
  20    0009.7c55.7a01    DYNAMIC     Gig0/1
S1>
```

🧪 Part 3 — Deploy MIME Server

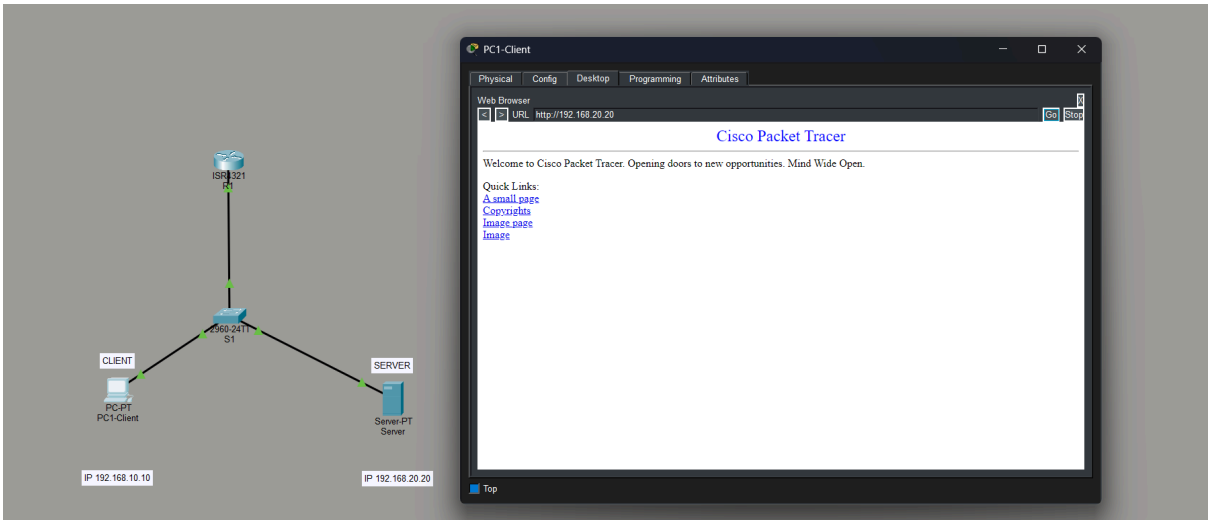
On Server (PC2)

```
None
cd automation
python server/main_enhanced.py --host 0.0.0.0 --port 65432 --verbose
```

On Client (PC1)

```
None
cd automation
python client/main_enhanced.py assets/sample.png --host 192.168.20.20 --port
65432
```

Confirm file saved.



📡 Part 4 — Wireshark Capture Walkthrough on PC1 and PC1

Step 1 — Start Capture

Open Wireshark.

Select interface connected to network.

Apply capture filter:

```
None
tcp port 65432
```

Step 2 — Initiate Transfer

Send file again.

Stop capture after completion.

Step 3 — Analyze Packets

A — ARP Exchange

Look for:

- Who has 192.168.20.20?

Confirm MAC resolution.

B — TCP Three-Way Handshake

Identify:

1 SYN 2 SYN-ACK 3 ACK

Check sequence numbers.

C — Data Transfer

Follow stream:

Right click → Follow → TCP Stream.

You should see JSON header similar to:

```
None
{"mime_type":"image/png","size":12345}
```

Followed by binary.

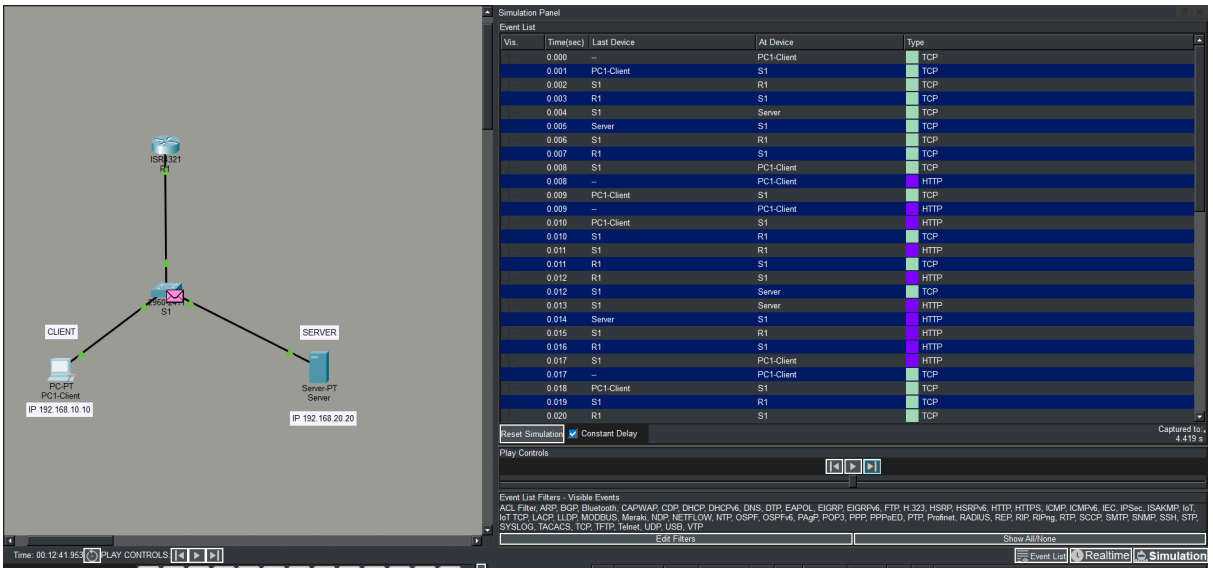
D — VLAN Tags

If capturing on trunk:

Look for:

```
None
802.1Q Virtual LAN
```

Note VLAN ID.



🔬 Packet Dissection Guide

Layer	Field	What to Observe
Ethernet	Src/Dst MAC	Frame direction
802.1Q	VLAN ID	Segmentation
IP	TTL	Routing behavior

Layer	Field	What to Observe
TCP	Flags	Reliability
Payload	JSON	MIME metadata

🛠️ Part 5 — Troubleshooting Labs

Scenario 1 — Broken Routing

Shutdown subinterface.

Observe:

- Ping failure
- TCP timeout

Scenario 2 — Firewall Block

Block port.

Observe:

- SYN retransmissions

Scenario 3 — Wrong Gateway

Observe:

- ARP incomplete

Scenario 4 — Packet Loss Simulation

Drop packets.

Observe:

- TCP retransmissions

- Window adjustment

📝 Lab Report Requirements

Include:

- Screenshots of configurations
- Ping results
- Wireshark capture screenshots
- JSON header evidence
- Encapsulation explanation
- Troubleshooting notes

🎓 Assessment Questions

1️⃣ Why does TCP retransmit lost segments?

เพราะ TCP เป็น reliable protocol ถ้าไม่ได้รับ ACK ภายใน timeout จะ retransmit segment เพื่อรับประกันว่า data ครบถ้วน

2️⃣ How does VLAN tagging affect broadcast domains?

VLAN แยก broadcast domain เครื่องใน VLAN 10 จะไม่ broadcast ไป VLAN 20 ลด broadcast traffic และเพิ่ม security

---

3️⃣ What role does MIME play compared to TCP headers?

- TCP header → จัดการ reliability (sequence, ACK, port)
- MIME → ให้ semantic meaning (file type, size)
  TCP ไม่รู้ว่าไฟล์คืออะไรMIME ทำให้ application เข้าใจข้อมูล

4️⃣ Where does routing decision occur?

Routing decision เกิดที่ Routerใน Network Layer (IP layer)ดู destination IP แล้วเลือก next hop

---

5️⃣ Why does ARP precede TCP communication?ก่อนส่ง

TCP segmentต้องรู้ MAC address ของปลายทางหรือ gateway ARP ใช้แปลง IP → MAC จึงต้องเกิดก่อน TCP data transfer

---

📊 Reflection Exercise

Explain:

How does application meaning survive multiple encapsulation layers?

Application meaning survives because:

1. แต่ละ layer treat upper-layer data เป็น opaque payload
2. ไม่มี layer ใดแก้ไข semantic content
3. TCP รับประกัน reliability
4. Decapsulation คืนข้อมูลเหมือนเดิม

แม้ข้อมูลจะถูก encapsulate หลายชั้น
แต่ meaning ไม่ถูกเปลี่ยน
เพียงแค่ถูกห่อด้วย control information

⚠️ Common Pitfalls

Server bind localhost → client ต่อไม่ได้

Missing trunk → VLAN ข้ามไม่ได้

Wrong VLAN → Ping fail

OS firewall → TCP timeout

Capture wrong interface → ไม่เห็น packet

🚀 Extension Activities

Add TLS encryption

เพิ่มความปลอดภัย
Payload จะถูกเข้ารหัส

Measure throughput vs latency

วัด performance
ดู TCP window effect

Capture checksum errors

ดูความผิดพลาดใน frame

Add checksum field to protocol

เพิ่ม integrity check ใน application layer

Containerize server

ใช้ Docker → ศึกษา distributed system

Compare with HTTP transfer

ดูความแตกต่างระหว่าง custom protocol กับ HTTP

🧬 Instructor Notes — What This Lab Really Teaches
This is not just networking.

1️⃣ Protocol layering philosophy

แต่ละ layer มีหน้าที่ชัดเจน

2 Reliability engineering

TCP retransmission / flow control

3 Metadata semantics

MIME ทำให้ข้อมูลมีความหมาย

4 Distributed system fundamentals

Client ↔ Server ผ่านหลาย layer